

NETWORK LOAD BALANCER

Submitted By

Syed Muhammad Huzaifa

22k-4450

Syed Umer Razi

22k-4451

.

.

**Department of Computer Science
National University of Computer & Emerging Sciences**

1. Motivation

With the rise of distributed web applications, ensuring high availability, fault tolerance, and efficient resource usage has become critical. Traditional load balancers like HAProxy and NGINX provide powerful features, but lack user-friendly, real-time configuration mechanisms. This project is motivated by the need to bridge that usability gap by offering a dynamic, interactive interface that empowers users—especially students and small organizations—to experiment with and understand load balancing strategies without deep system-level configuration.

The project was conceived to:

- *Educate students on how modern load balancers operate under the hood.*
- *Demonstrate real-world network traffic distribution using open-source tools.*
- *Empower users to switch algorithms.*
- *Simplify HAProxy configuration management through a web-based control layer.*

2. Overview

Specify the core idea behind the project. This should include

2.1 Significance of the Project

This project presents a practical and scalable network load balancing system combining NGINX, HAProxy, and Flask. The primary motivation is to provide a customizable and easily extendable load balancer that supports dynamic reconfiguration via a user interface. The system is built with open-source tools and targets small to mid-sized organizations or students wishing to understand modern load balancing architectures. It provides real-time algorithm switching, server monitoring, and backend integration in an educational and demonstrative setting.

2.2 Description of the Project

The project integrates three key components: NGINX as the entry point to handle client requests, HAProxy as the primary load balancer distributing requests to multiple Flask backend servers, and a Flask-based UI to monitor and control the load balancing algorithms dynamically. Users can switch between Round Robin, Least Connections, and IP Hashing algorithms directly from the web UI. The system reflects changes by updating the HAProxy configuration file and reloading HAProxy to apply the changes. Although the current implementation requires manual sudo password entry to reload HAProxy, the project lays the foundation for future automation using secure privilege escalation methods.

2.3 Background of the Project

Modern web applications require scalability and high availability. Load balancers distribute incoming traffic among multiple servers to ensure no single server is overwhelmed, thereby improving performance and reliability. Traditional load balancers like NGINX and HAProxy are powerful but require manual configuration. This project introduces a dynamic control layer on top of HAProxy using Flask, allowing algorithm switching at runtime. This bridges the gap between static configurations and real-time adaptability, offering both learning and practical benefits.

- Nginx Load Balancing: <https://www.nginx.com/resources/glossary/load-balancing/>
- Python Socket Programming: <https://docs.python.org/3/howto/sockets.html>
- Flask Documentation: <https://flask.palletsprojects.com/>
- HAProxy Guide: <https://www.haproxy.com/documentation/>.
- https://github.com/MG-Musty/load-balancer-with-nginx_web_server

2.4 Project Category

This is a **Product-based** academic project with partial alignment to research and industry practices, demonstrating real-world load balancing principles in an interactive and extendable manner.

3. Features / Scope / Modules

- **Real-Time Load Balancing Algorithm Switching:** Enables switching between Round Robin, Least Connections, and IP Hashing algorithms directly from the UI.
- **Dynamic HAProxy Configuration Reload:** Updates HAProxy configuration and reloads the service without restarting the system.
- **Flask-Based Web UI:** Provides a dashboard for monitoring current backend servers and switching algorithms.
- **Health Monitoring Support:** Includes backend server health checks in the HAProxy configuration to ensure traffic is only routed to healthy servers.
- **Modular Architecture:** Separation of concerns between NGINX (entry), HAProxy (load distribution), and Flask (UI/control logic).
- **Authentication-Protected HAProxy Stats Interface:** HAProxy dashboard at port 8404 provides password-protected access to internal statistics and the current balancing algorithm.
- **Scalability:** Easily extendable by adding/removing backend servers or integrating support for launching/stopping containers.

4. Project Planning

TASK	DESCRIPTION	RESPONSIBILITY	TIMELINE
Requirement Gathering	Define Components and flow	All Members	Week 1
Flask UI Development	Create dashboard with forms and switches	Huzaifa	Week 2-3
NGINX + HAProxy Setup	Configure server and load balancer	Umer	Week 2
Integration	Link all components into one system	All Members	Week 4
Testing and Debugging	Test UI , switching, reload, request flow	All Members	Week 5
Report Writing	Final Documentation	Huzaifa	Week 6

5. Project Feasibility

● Technical Feasibility

The project is **technically feasible** due to the use of well-established, open-source technologies such as **HAProxy**, **NGINX**, and **Flask**. These tools are known for their stability, performance, and strong community support.

- **HAProxy** is widely used in production-grade environments for load balancing and supports real-time configuration reloads without downtime.
- **NGINX** serves as a reliable reverse proxy and web server, commonly used at the frontend in high-performance systems.
- **Flask** provides a lightweight yet powerful framework to build the UI and control logic for dynamic interaction with HAProxy.

Technical risks are minimal but may include:

- Incorrect HAProxy configuration syntax causing reload failures.
- Permission issues due to requiring `sudo` access to reload HAProxy.
- Challenges in synchronizing live UI changes with real-time HAProxy behavior.

● Economic Feasibility

The project has **high economic feasibility** because it:

- Relies entirely on **open-source software**, with **no licensing costs** involved.
- Can be developed and deployed on **low-cost hardware**, such as personal laptops or virtual machines with basic specifications.
- Requires only a modest investment in time and learning resources, making it ideal for student and academic purposes.

This makes the project suitable even for organizations with limited IT budgets or educational institutions focusing on practical demonstrations.

- **Schedule Feasibility**

The project follows a **realistic and achievable timeline**, broken into modular tasks executed in parallel by the team members. Tasks are distributed based on skillset and dependencies, reducing bottlenecks.

For example:

- While one member develops the Flask-based UI, another sets up and configures NGINX and HAProxy.
- Integration and testing happen after the components are individually stable.
- Weekly milestones help track progress and ensure timely delivery.

The schedule reflects the actual availability and capability of the team members, with a buffer included for debugging and documentation.

6. Hardware and Software Requirements

Hardware

1. Minimum 4-core CPU

- Why: Load balancing involves handling multiple concurrent network requests, often in parallel. A quad-core processor ensures the system can process incoming traffic, update the HAProxy configuration, and serve backend responses without lag.
- Benefit: Allows concurrent handling of UI interactions, backend services, and configuration reloads efficiently.

2. 8GB RAM

- Why: Multiple processes (NGINX, HAProxy, Flask backend servers, and the Flask web UI) will be running simultaneously. Flask also uses memory for routing logic and template rendering.
- Benefit: Prevents performance bottlenecks and ensures smooth switching of algorithms and real-time updates.

3. Network Interface (for client-server simulation)

- Why: Required to simulate real-world HTTP request flow between the client, NGINX, HAProxy, and backend servers.
- Benefit: Allows internal routing and monitoring of traffic among services on different ports or even across containers/VMs.

Software

1. OS: Ubuntu/Linux

- Why: Linux-based systems (especially Ubuntu) offer better native support for networking tools like HAProxy and NGINX, and are often used in production environments.
- Benefit: Simplifies installation, configuration, and scripting (e.g., bash scripts to reload services).

2. NGINX

- Why: Serves as the entry point to handle incoming client requests before forwarding them to HAProxy.
- Benefit: Can cache static content, terminate SSL (if used), and offload preliminary request filtering before load balancing.

3. HAProxy

- Why: Core load balancer that distributes incoming requests to multiple Flask backend servers based on the selected algorithm.
- Benefit: Robust, high-performance software with support for dynamic backend management and monitoring.

4. Python 3.8+

- Why: The Flask web server and control panel are written in Python. Version 3.8+ ensures compatibility with most modern Flask and third-party libraries.
- Benefit: Provides features like async support and improved performance over older versions.

5. Flask

- Why: Lightweight Python web framework used to create the monitoring UI and configuration control interface.
- Benefit: Easy to develop and extend; supports routing, form handling, and template rendering.

6. sudo privileges for reloading HAProxy

- Why: Reloading HAProxy (sudo systemctl reload haproxy) is required every time the configuration file is modified.
- Benefit: Ensures that changes made via the UI are applied in real time to the load balancer.

7. Browser for Ui access

- Why: The Flask Web UI is accessed through a browser by visiting localhost:5000 (or another port).
- Benefit: Provides a graphical interface for users to switch algorithms and monitor servers

7. Project Working demo:

1. Starting HAProxy:

```
huzaiifa@huzaiifa-virtual-machine:~/Desktop$ cd Network_load_balancer(FINAL1)\)/
huzaiifa@huzaiifa-virtual-machine:~/Desktop/Network_load_balancer(FINAL1)$ cd haproxy
huzaiifa@huzaiifa-virtual-machine:~/Desktop/Network_load_balancer(FINAL1)/haproxy$ sudo haproxy -f haproxy.cfg
[sudo] password for huzaiifa:
Sorry, try again.
[sudo] password for huzaiifa:
```

2. Starting NGINX:

```
huzaiifa@huzaiifa-virtual-machine:~/Desktop/Network_load_balancer(FINAL1)/nginx$ sudo nginx -c $PWD/nginx.conf
huzaiifa@huzaiifa-virtual-machine:~/Desktop/Network_load_balancer(FINAL1)/nginx$ sudo lsof -i :8080
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
nginx 4106 root 6u IPv4 37360 0t0 TCP *:http-alt (LISTEN)
nginx 4107 nobody 6u IPv4 37360 0t0 TCP *:http-alt (LISTEN)
huzaiifa@huzaiifa-virtual-machine:~/Desktop/Network_load_balancer(FINAL1)/nginx$
```

3. Starting backend servers:

```
huzaiifa@huzaiifa-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1)/backend_servers
huzaiifa@huzaiifa-virtual-machine:~/Desktop/Network_load_balancer(FINAL1)$ cd backend_servers/
huzaiifa@huzaiifa-virtual-machine:~/Desktop/Network_load_balancer(FINAL1)/backend_servers$ python3 app1.py
* Serving Flask app 'app1'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.12.128:5001
Press CTRL+C to quit
127.0.0.1 - - [04/May/2025 09:25:09] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:11] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:13] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:17] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:19] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:19] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:21] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:23] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:27] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:33] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:35] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:37] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:39] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:41] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:43] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:45] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:49] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:53] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:55] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:57] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:02] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:06] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:08] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:10] "GET / HTTP/1.1" 200 -
```

```
huzailfa@huzailfa-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1)/backend_servers$ python3 app2.py
* Serving Flask app 'app2'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on https://192.168.12.128:5002
Press CTRL+C to quit
127.0.0.1 - - [04/May/2025 09:25:19] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:21] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:23] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:27] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:30] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:33] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:35] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:37] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:39] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:41] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:43] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:45] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:49] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:53] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:55] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:57] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:02] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:06] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:08] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:14] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:18] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:22] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:24] "GET / HTTP/1.1" 200 -
```

```
huzailfa@huzailfa-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1)/backend_servers$ python3 app3.py
* Serving Flask app 'app3'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5003
* Running on http://192.168.12.128:5003
Press CTRL+C to quit
127.0.0.1 - - [04/May/2025 09:25:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:27] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:33] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:35] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:37] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:39] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:41] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:43] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:45] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:49] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:53] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:55] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:57] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:02] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:06] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:08] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:14] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:18] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:22] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:24] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:26] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:28] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:30] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:32] "GET / HTTP/1.1" 200 -
```

```
huzai@huzai-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1))/backend_servers
huzai@huzai-virtual-machine:~/Desktop/Network_load_balancer(FINAL1))/backend_servers$ python3 app4.py
* Serving Flask app 'app4'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5004
* Running on http://192.168.12.128:5004
Press CTRL+C to quit
127.0.0.1 - - [04/May/2025 09:25:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:33] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:35] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:37] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:39] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:40] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:41] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:43] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:45] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:49] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:49] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:53] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:55] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:57] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:02] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:06] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:08] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:09] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:14] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:14] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:18] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:22] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:24] "GET / HTTP/1.1" 200 -
```

```
huzai@huzai-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1))/backend_servers
huzai@huzai-virtual-machine:~/Desktop/Network_load_balancer(FINAL1))/backend_servers$ python3 app5.py
python3: can't open file '/home/huzai/Desktop/Network_load_balancer(FINAL1))/backend_servers/app': [Errno 2] No such file or directory
huzai@huzai-virtual-machine:~/Desktop/Network_load_balancer(FINAL1))/backend_servers$ python3 app5.py
* Serving Flask app 'app5'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5005
* Running on http://192.168.12.128:5005
Press CTRL+C to quit
127.0.0.1 - - [04/May/2025 09:25:37] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:39] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:41] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:43] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:45] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:49] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:53] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:55] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:25:57] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:02] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:06] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:08] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:14] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:18] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:22] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:24] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:26] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:28] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:30] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:32] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:34] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:36] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:38] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:41] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:26:43] "GET / HTTP/1.1" 200 -
```

4. Starting the FLASK_ui:

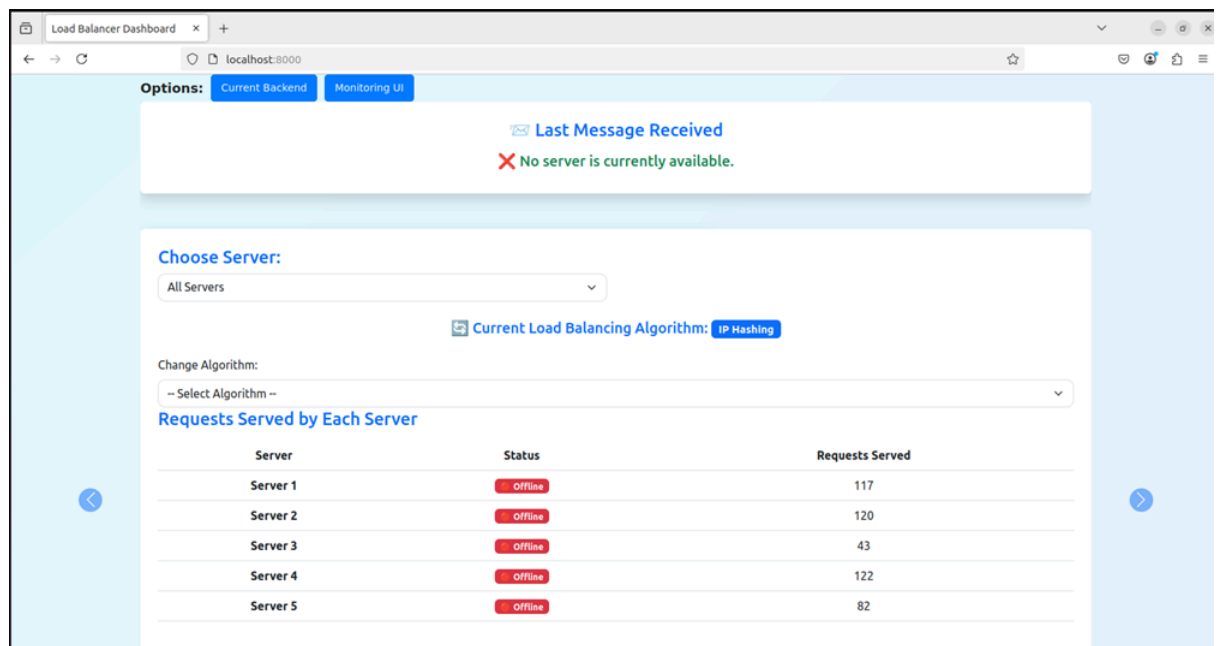
```
huzaiifa@huzaiifa-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1)$ cd load_balancer_ui/
huzaiifa@huzaiifa-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1)/load_balancer_ui$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 780-908-190
127.0.0.1 - - [04/May/2025 09:22:32] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:32] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [04/May/2025 09:22:32] "GET /logs HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:33] "GET /status HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:33] "GET /counter HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:33] "GET /counter HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:33] "GET /last_response HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:33] "GET /last_response HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:34] "GET /counter HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:34] "GET /counter HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:34] "GET /last_response HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:34] "GET /status HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:35] "GET /last_response HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:36] "GET /counter HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:36] "GET /last_response HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:36] "GET /status HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:36] "GET /counter HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:37] "GET /last_response HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:38] "GET /counter HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:38] "GET /last_response HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:38] "GET /status HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:38] "GET /counter HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:39] "GET /last_response HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:40] "GET /counter HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:40] "GET /last_response HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:40] "GET /counter HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2025 09:22:40] "GET /status HTTP/1.1" 200 -
```

5. Starting the haproxy_log:

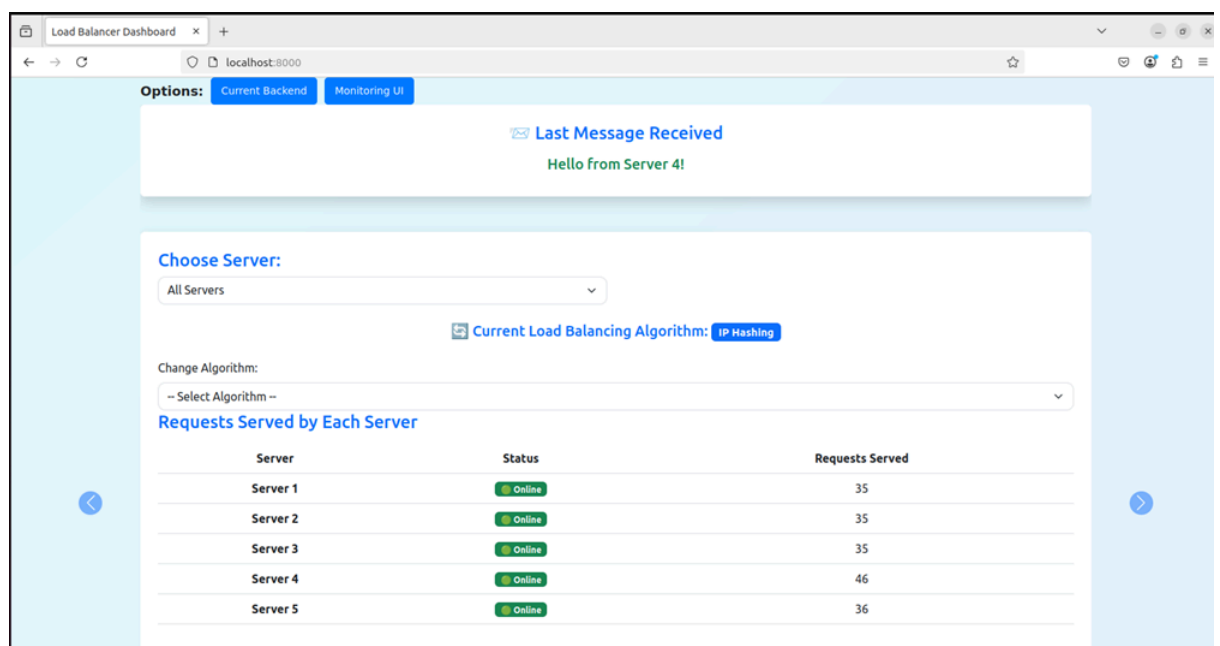
```
huzaiifa@huzaiifa-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1)/logs
huzaiifa@huzaiifa-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1)/backend_servers$ cd .
huzaiifa@huzaiifa-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1)/backend_servers$ cd ..
huzaiifa@huzaiifa-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1)$ cd logs/
huzaiifa@huzaiifa-virtual-machine: ~/Desktop/Network_load_balancer(FINAL1)/logs$ sudo socat -t0 -u UNIX-RECVD:/tmp/haproxy-log.sock STDOUT | tee -a access.log
[sudo] password for huzaiifa:
<134>May 4 09:26:00 haproxy[4059]: Connect from 127.0.0.1:50592 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:26:09 haproxy[4059]: Connect from 127.0.0.1:43924 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:26:14 haproxy[4059]: Connect from 127.0.0.1:43944 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:26:15 haproxy[4059]: Connect from 127.0.0.1:43946 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:26:20 haproxy[4059]: Connect from 127.0.0.1:51848 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:26:24 haproxy[4059]: Connect from 127.0.0.1:51864 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:26:25 haproxy[4059]: Connect from 127.0.0.1:51876 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:26:35 haproxy[4059]: Connect from 127.0.0.1:42292 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:26:41 haproxy[4059]: Connect from 127.0.0.1:51212 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:26:50 haproxy[4059]: Connect from 127.0.0.1:33048 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:26:56 haproxy[4059]: Connect from 127.0.0.1:33056 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:27:00 haproxy[4059]: Connect from 127.0.0.1:39018 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:27:06 haproxy[4059]: Connect from 127.0.0.1:39042 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:27:15 haproxy[4059]: Connect from 127.0.0.1:53980 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:27:25 haproxy[4059]: Connect from 127.0.0.1:46142 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:27:31 haproxy[4059]: Connect from 127.0.0.1:35336 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:27:36 haproxy[4059]: Connect from 127.0.0.1:35356 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:27:45 haproxy[4059]: Connect from 127.0.0.1:53960 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:27:46 haproxy[4059]: Connect from 127.0.0.1:53962 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:27:50 haproxy[4059]: Connect from 127.0.0.1:50622 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:28:00 haproxy[4059]: Connect from 127.0.0.1:39274 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:28:11 haproxy[4059]: Connect from 127.0.0.1:55986 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:28:15 haproxy[4059]: Connect from 127.0.0.1:55992 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:28:36 haproxy[4059]: Connect from 127.0.0.1:41458 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:28:40 haproxy[4059]: Connect from 127.0.0.1:39298 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:28:51 haproxy[4059]: Connect from 127.0.0.1:37342 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:28:55 haproxy[4059]: Connect from 127.0.0.1:37350 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:28:56 haproxy[4059]: Connect from 127.0.0.1:37354 to 127.0.0.1:80 (http_front/HTTP)
<134>May 4 09:29:00 haproxy[4059]: Connect from 127.0.0.1:52644 to 127.0.0.1:80 (http_front/HTTP)
```

6. UI screenshots:

Before Starting Servers:

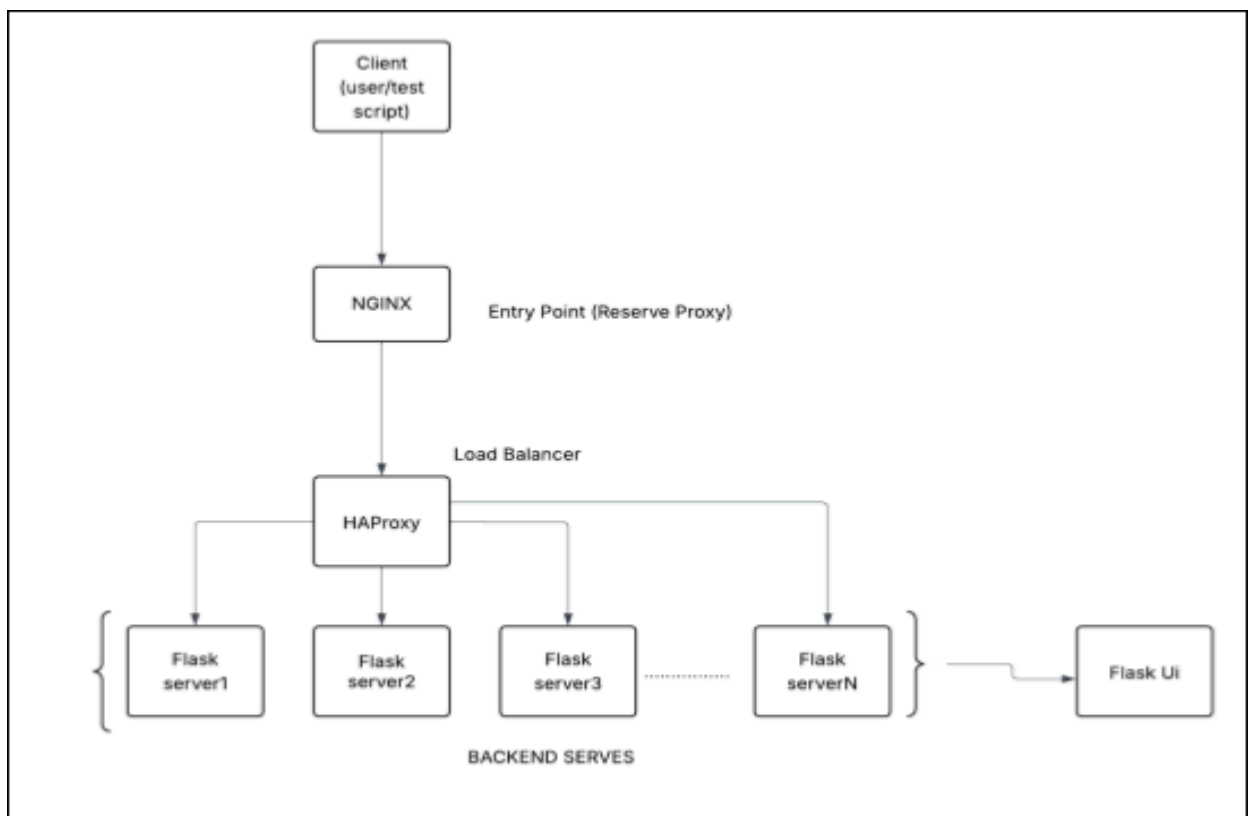


After Starting Servers:



8. Diagrammatic Representation of the Overall System

Figure 1. Flow of HTTP requests from Client to Backend via NGINX and HAProxy.



9. References

- [1] HAProxy Technologies, "HAProxy Documentation", <https://www.haproxy.org/>
- [2] NGINX Inc., "NGINX Documentation", <https://docs.nginx.com/>
- [3] Flask, "The Pallets Projects", <https://flask.palletsprojects.com/>
- [4] DigitalOcean, "How to Set Up HAProxy with SSL on Ubuntu", <https://www.digitalocean.com/community/tutorials/>
- [5] Official Ubuntu Documentation, <https://help.ubuntu.com/>

[6] Stack Overflow discussions and GitHub issues related to dynamic HAProxy reconfiguration.

[7] <https://github.com/haproxy/haproxy>